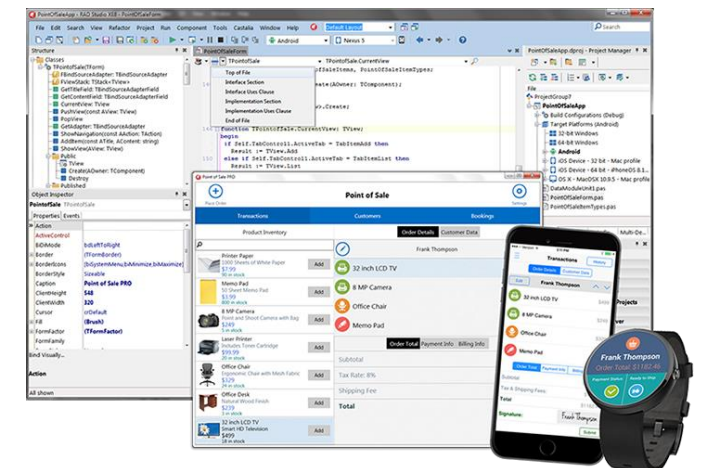
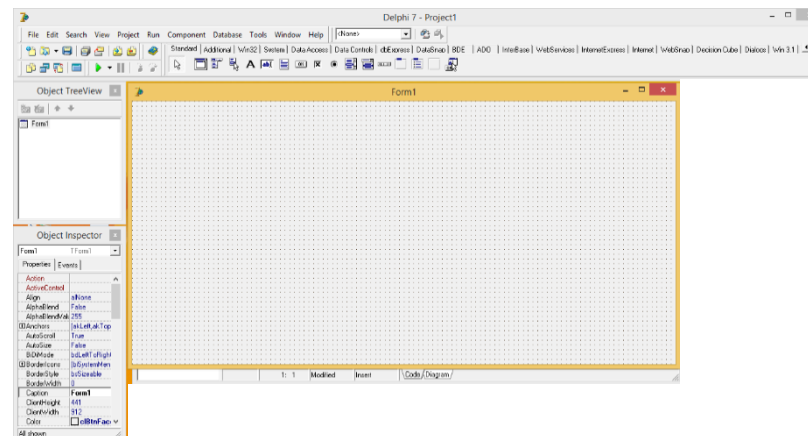
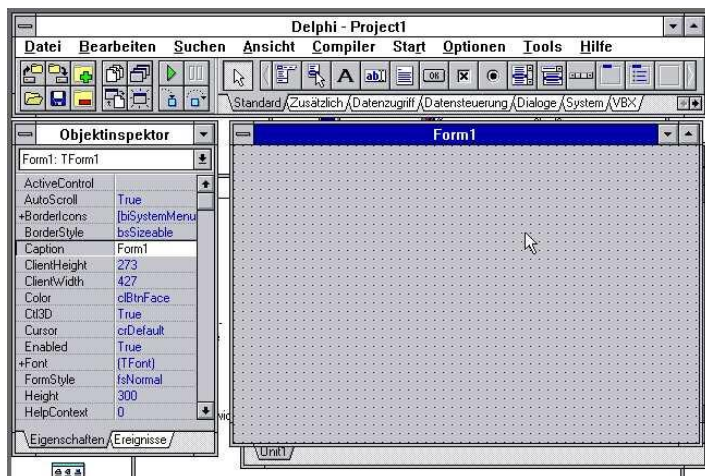


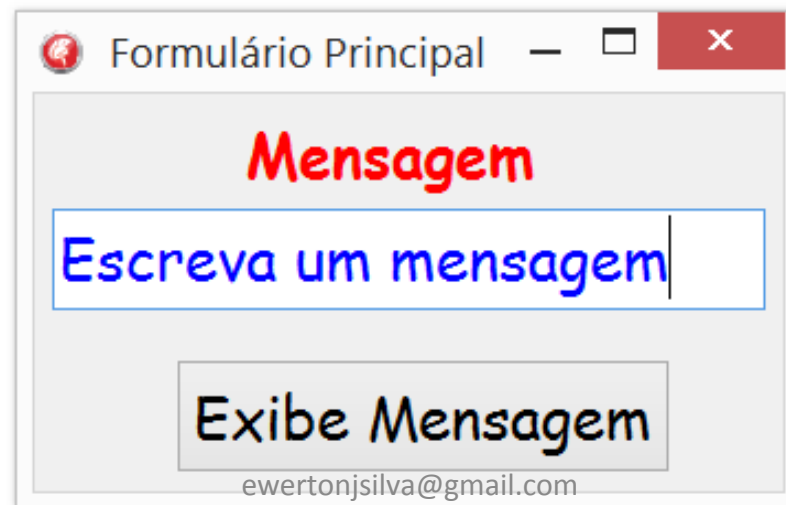
INTRODUÇÃO DELPHI

Ewerton J. Silva



Programando em Delphi – Ex 1

- Já sabemos como criar um executável, salvar e abrir um programa, renomear componentes e definir como eles serão visualizados, mas nosso programa não faz nada.
- A ideia desta janela é fazer com que ao clicar no botão exibe mensagem, o programa exiba o conteúdo da caixa de texto no Label acima da mesma.
- Para que isto seja possível precisamos programar...



Palavras reservadas

As palavras reservadas independente de qualquer biblioteca, fazem parte do núcleo da linguagem, e por isso são utilizadas por qualquer compilador Delphi. Normalmente elas estão em evidência no programa (**negrito**).

```
procedure TfrmPrograma.btnMensagemClick(Sender: TObject);  
begin  
  
    if (edtMensagem.Text = '') then  
    begin  
        lblMensagem.Caption := 'Você não digitou nada';  
    end  
    else  
    begin  
        lblMensagem.Caption := edtMensagem.Text;  
        edtMensagem.Text := '';  
    end;  
  
end;
```

Palavras reservadas

Palavra	Descrição
Begin	Início de bloco de programa.
End	Fim de Bloco de Programa.
Type	Definição de tipos.
Var	Definição de variáveis. Uses Definição de bibliotecas.
Class	Definição de classes.
Implementation	Seção da unidade, onde estão todas as definições da mesma, variáveis, tipos, constantes, bibliotecas, etc., que são privadas à unidade. Além disso está todo o código dos métodos da unidade.
Interface	Seção da unidade, onde estão todas as definições da mesma, variáveis, tipos, constantes, bibliotecas, etc., que serão tornadas públicas pela unidade.

Palavras reservadas

Palavra	Descrição
Do	Comando interno de execução de algum outro comando.
while	Comando de repetição.
for	Comando de repetição.
if	Comando de seleção.
else	Comando de exceção a uma seleção.
Case	Comando de seleção.
Then	Comando de auxiliar de seleção.
Public	Propriedades e Métodos que podem ser acessados por qualquer unidade do programa.
Private	Propriedades e Métodos que só podem ser acessados pela própria unidade.

Palavras reservadas

Palavra	Descrição
Protected	Propriedades e Métodos que podem ser acessados apenas pela própria unidade e pelos métodos das unidades derivadas desta.
Unit	Chave para a declaração do nome da unidade.
Array	Palavra para a definição de vetores.
Of	Comando auxiliar para definições de tipos.
Repeat	Comando de Repetição.
Until	Comando auxiliar na repetição.
With	Comando utilizado para referir-se a um registro de um objeto de forma direta.

Lista completa de palavras reservadas

A *Object Pascal* se utiliza das seguintes palavras reservadas, não podendo as mesmas serem utilizadas ou redefinidas:

And	Exports	Library	Set
Array	File	Mod	Shl
As	Finally	Nil	Shr
Asm	For	Not	String
Begin	Function	Object	Then
Case	Goto	Of	To
Class	If	On	Try
Const	Implementation	Or	Type
Constructor	In	Packed	Unit
Destructor	Inherited	Procedure	Until
Div	Initialization	Program	Uses
Do	Inline	Property	Var
Downto	Interface	Raise	While
Else	Is	Record	With
End	Label	Repeat	Xor
Except			

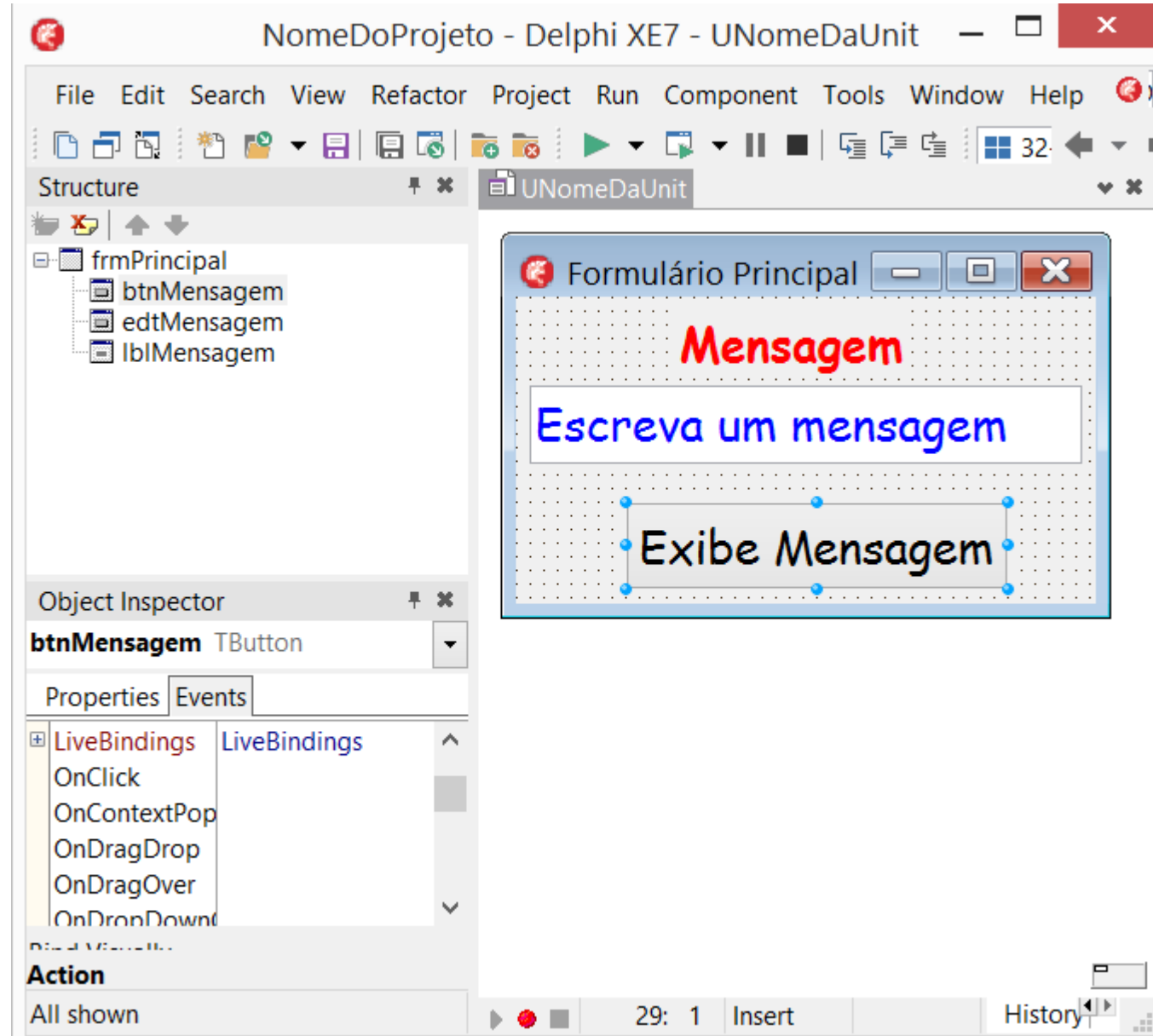
Lista completa de palavras reservadas

Uma outra lista a seguir, apresenta as diretivas que são utilizadas em contextos de identificação de objetos:

Absolute	Export	Name	Published
Abstract	External	Near	Read
Assembler	Far	Nodefault	Resident
At	Forward	Override	Stored
Cdecl	Index	Private	Virtual
Default	Interrupt	Protected	Write
Dynamic	Message	Public	

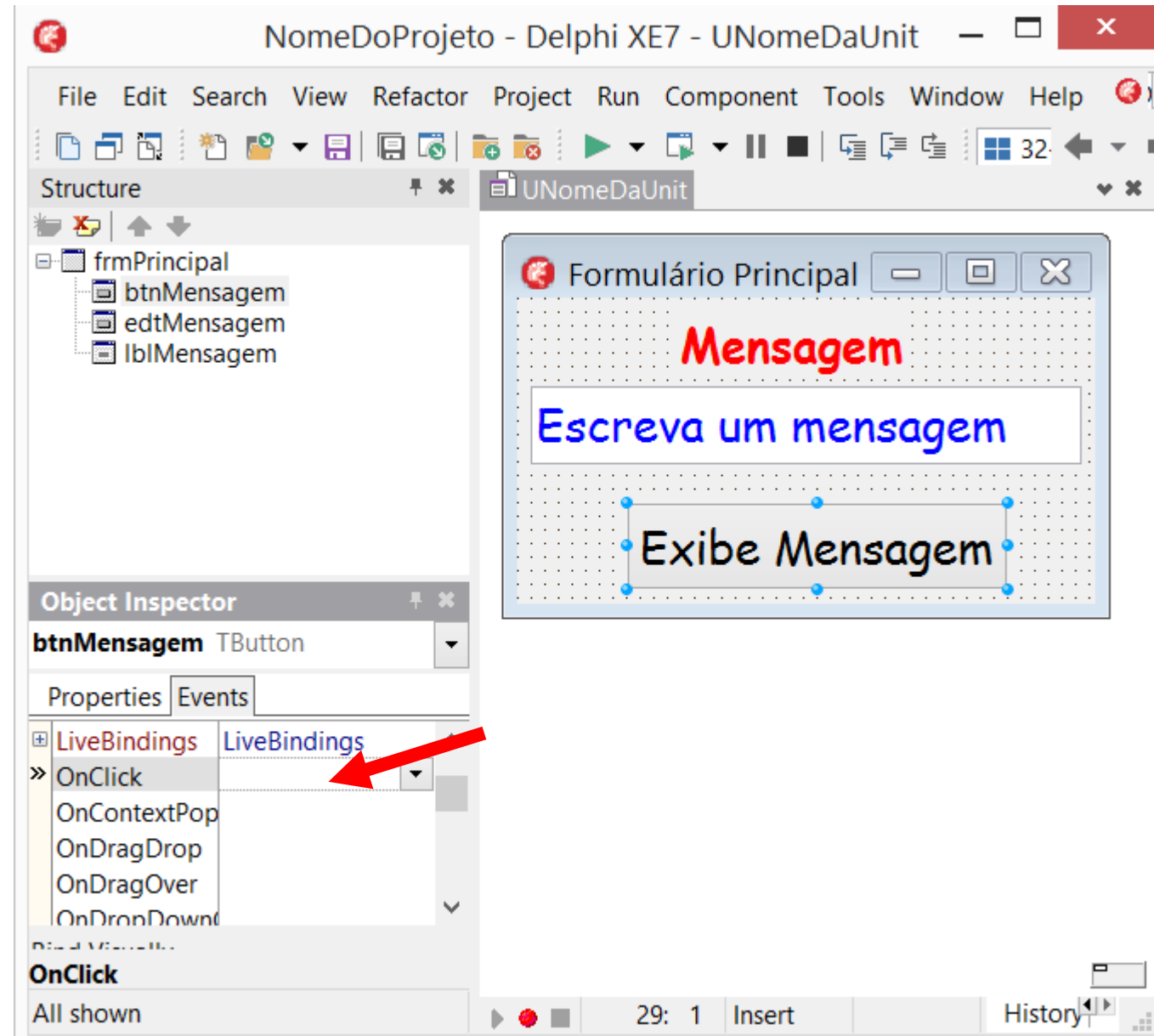
Primeiro código

- Nosso programa vai permitir que quando o usuário do sistema clicar no botão Exibe Mensagem, o conteúdo do Edit será exibido no Label.
- Para que isto aconteça devemos programar no evento OnClick do componente btnMensagem



Primeiro código

- A guia Events permite o desenvolvedor definir um manipulador de evento em Object Pascal para um determinado evento que pode ser disparado pelo usuário ou pelo sistema.
- Clique duas vezes na linha em frente ao evento **OnClick**...



Primeiro código

- Automaticamente a Unit ficará em evidência e uma procedure é criada.
- É dentro dela que iremos inserir a programação

```
UNomeDaUnit
unit UNomeDaUnit;
interface
uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs, Vcl.StdCtrls;
10 type
  TfrmPrincipal = class(TForm)
    lblMensagem: TLabel;
    edtMensagem: TEdit;
    btnMensagem: TButton;
    procedure btnMensagemClick(Sender: TObject);
  private
    { Private declarations }
  public
    { Public declarations }
20 end;

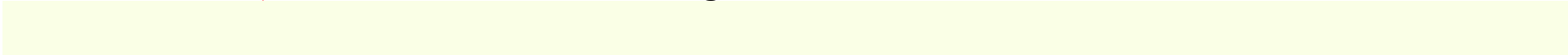
var
  frmPrincipal: TfrmPrincipal;

implementation
  {$R *.dfm}

30 procedure TfrmPrincipal.btnMensagemClick(Sender: TObject);
31 begin
  end;
end.
```

Programando no evento OnClick

- O código deve ser inserido entre o begin e o end;

	Nome do Form	Nome do Objeto / Componente	Evento
			
procedure	<code>TfrmPrincipal.btnMensagemClick(Sender: TObject);</code>		
begin	 Início do bloco de código referente evento		
			
end;	 Fim do bloco de código referente ao evento		

Programando no evento OnClick

- Insira o código a seguir e execute o programa.

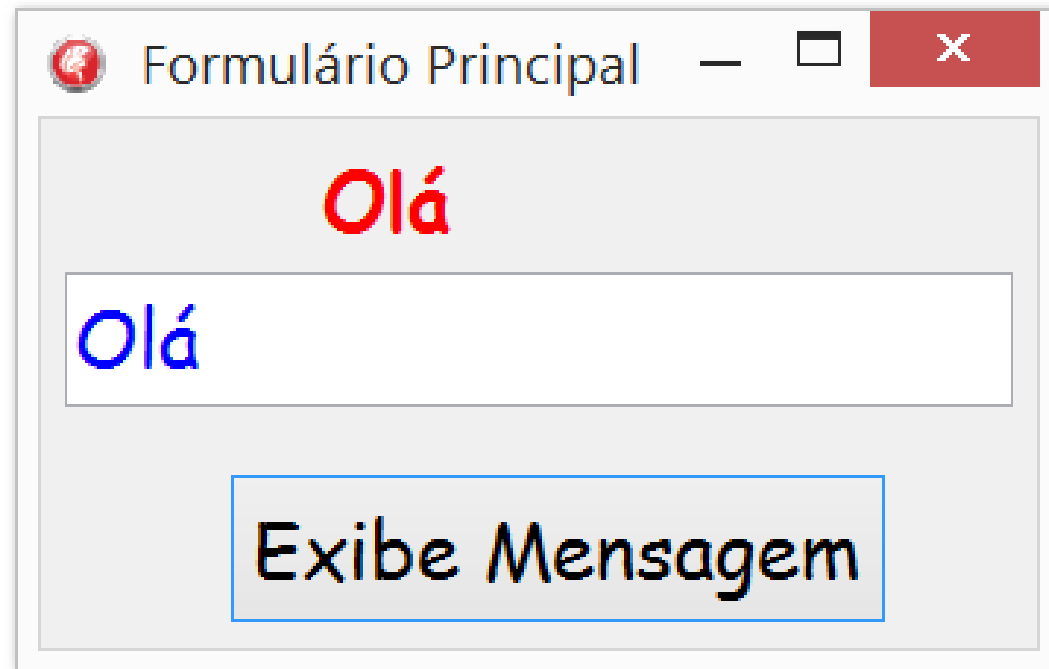
```
procedure TfrmPrincipal.btnMensagemClick(Sender: TObject);  
begin  
→ lblMensagem.Caption := edtMensagem.Text;  
end;
```



A propriedade **Caption** do componente **lblMensagem** recebe o valor contido na propriedade **Text** do componente **edtMensagem**

Programando no evento OnClick

- Escreva uma mensagem na caixa de texto e clique no botão “Exibe Mensagem” e veja o resultado.



Programando no evento OnClick

- Podemos inserir comentários para nos auxiliar a lembrar sobre o código inserido. Comentários não serão exibidos na execução do programa, mas para que não ocorra erros no código é necessário escreve-los corretamente.

```
• 30 procedure TfrmPrincipal.btnMensagemClick(Sender: TObject);  
•   begin  
•       // Ex: 1 - Comentário de 1 linha  
•  
•       { Ex: 2 - Comentário  
•         de mais de uma linha }  
•  
•       (* Ex: 3 - Comentário de  
•         mais de uma linha *)  
•  
•       lblMensagem.Caption := edtMensagem.Text;  
• 40 end;
```

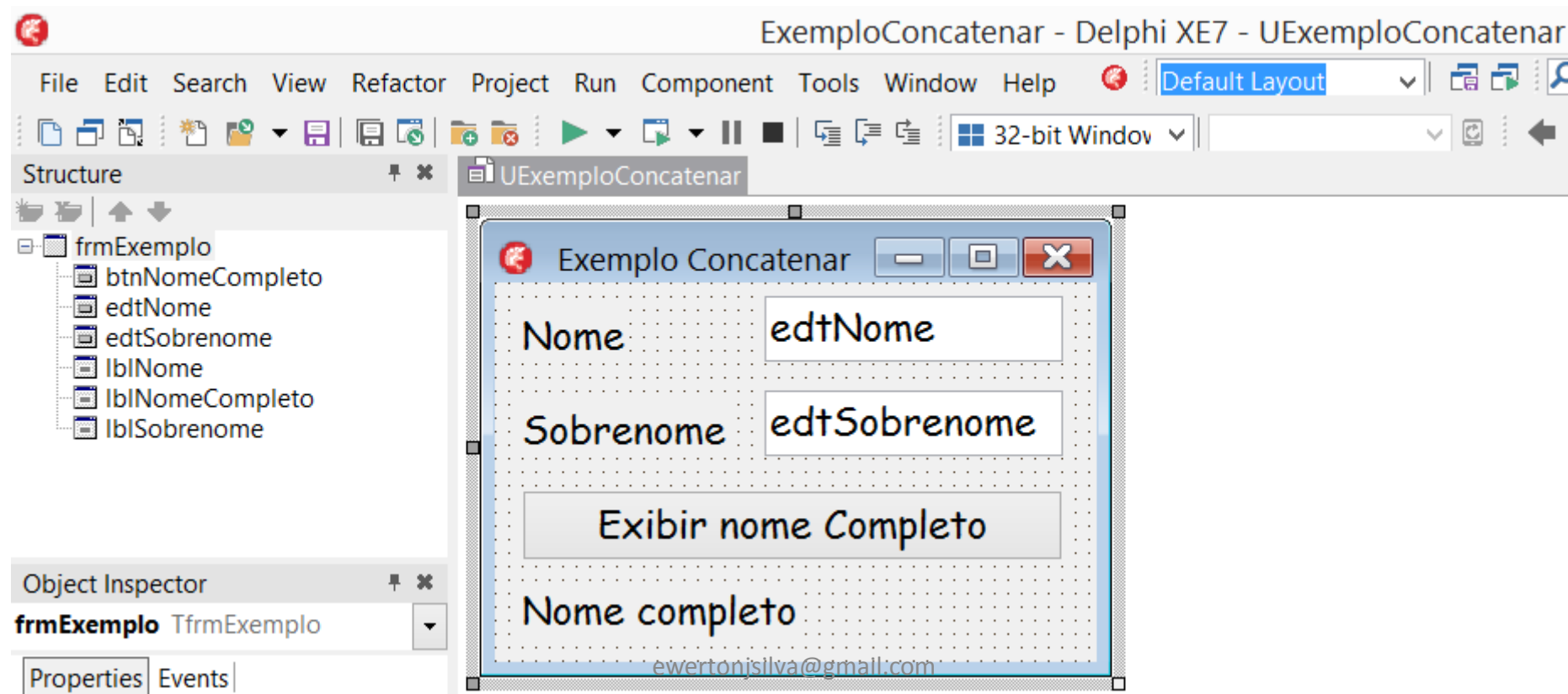
Programando no evento OnClick

É importante se habituar a sempre comentar o código, pois isto ajudará no aprendizado e é uma boa prática de programação. Escrever pensando em outros programadores que podem, no futuro, ter acesso ao código escrito por nós, seja para dar manutenção, seja para implementar algo. Ou mesmo que a própria pessoa que escreveu o código precise de manutenção e não lembre mais o porque de determinado bloco de programação, neste momento os comentários auxiliam muito.

```
procedure TfrmPrincipal.btnMensagemClick(Sender: TObject);  
begin  
    { A propriedade Caption do componente lblMensagem recebe o valor  
      contido na propriedade Text do componente edtMensagem }  
    lblMensagem.Caption := edtMensagem.Text;  
    // apaga o conteúdo do edtMensagem  
    edtMensagem.Clear;  
end;
```

Concatenando conteúdo dos Edits – Ex 2

Crie programa semelhante ao da imagem abaixo, utilizando os mesmos nomes do exemplo para os componentes, Unit, Form e o Projeto.



Concatenando conteúdo dos Edits

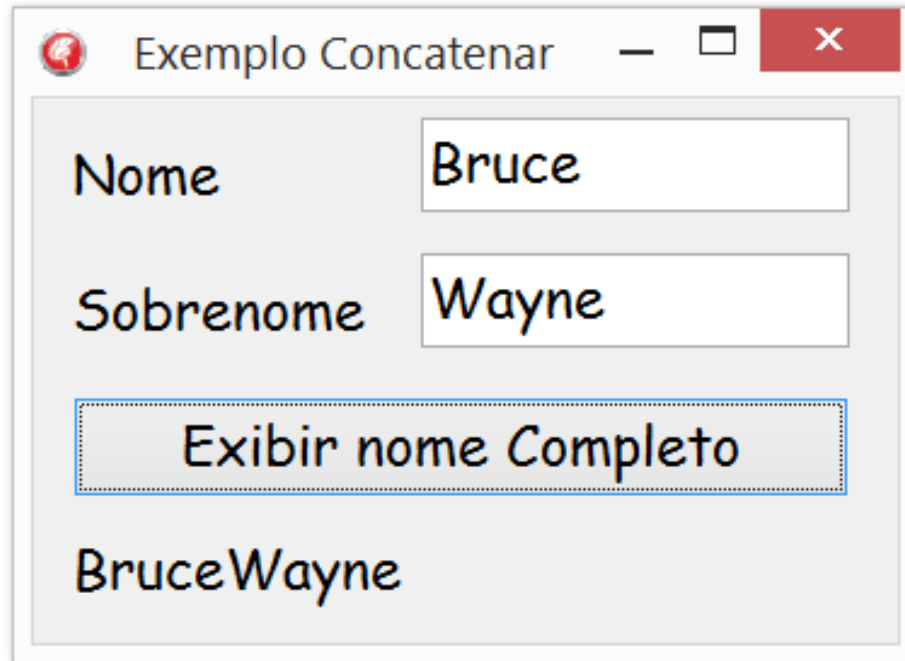
Insira o código abaixo no evento OnClick do btnNomeCompleto

Para concatenar os campos vamos utilizar o sinal de “+”

```
procedure TfrmExemplo.btnNomeCompletoClick(Sender: TObject);  
begin  
    // Combina o nome e o sobrenome digitados nos Edits  
    lblNomeCompleto.Caption := edtNome.Text + edtSobrenome.Text;  
end;
```

Concatenando conteúdo dos Edits

Neste exemplo o resultado foi um nome exibido sem espaços, vamos arrumar isto agora.



Exemplo Concatenar

Nome

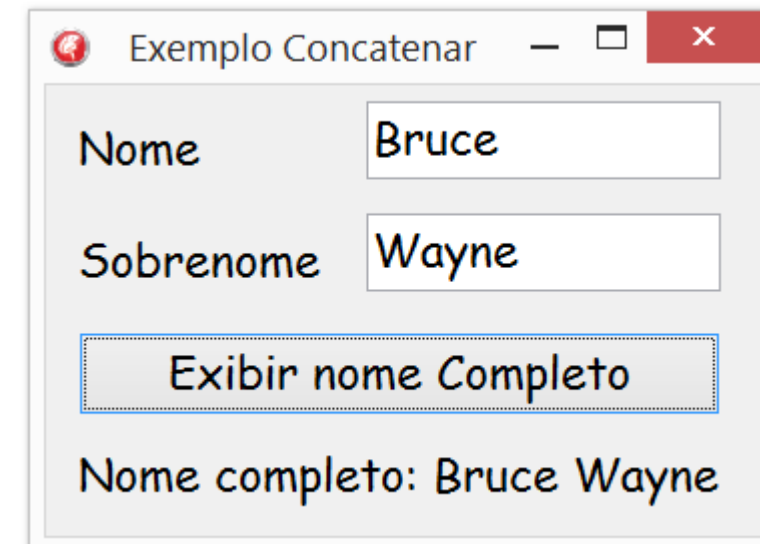
Sobrenome

BruceWayne

Concatenando conteúdo dos Edits

Insira o código abaixo para melhorar a exibição do conteúdo.

```
procedure TfrmExemplo.btnNomeCompletoClick(Sender: TObject);  
begin  
    // Combina o nome e o sobrenome digitados nos Edits  
    lblNomeCompleto.Caption := 'Nome completo: ' + edtNome.Text  
                               + ' ' + edtSobrenome.Text;  
  
    { Textos inseridos via código devem ser escritos entre aspas  
      simples }  
end;
```



Exemplo Concatenar

Nome Bruce

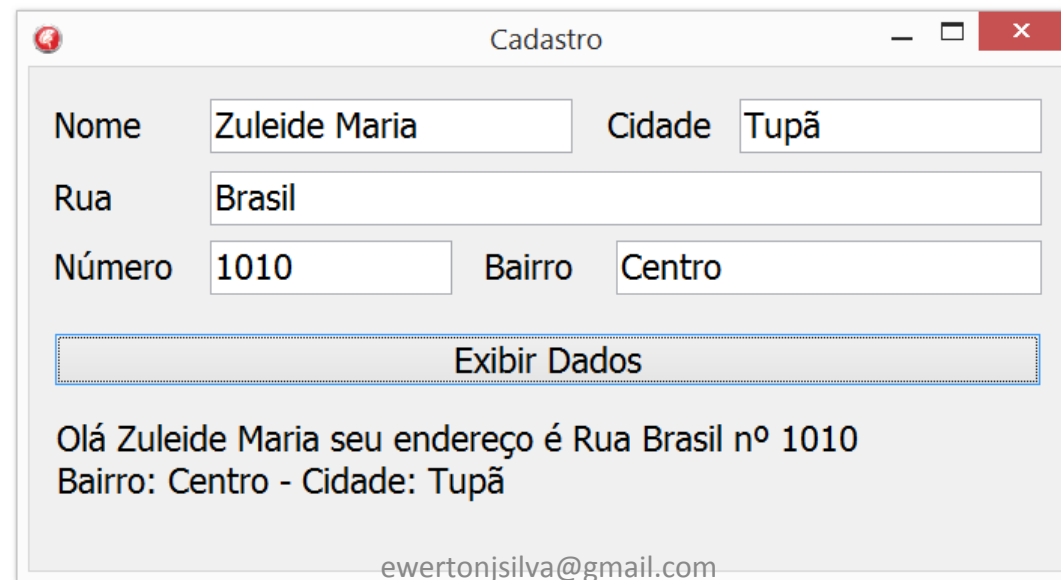
Sobrenome Wayne

Exibir nome Completo

Nome completo: Bruce Wayne

Exercícios

19. Acesse o link <http://www.devmedia.com.br/a-importancia-de-codigos-bem-escritos/25900> leia o artigo e responda o porque os códigos devem ser bem escritos.
20. Crie um programa que funcione como o da imagem abaixo.



The image shows a Java Swing window titled "Cadastro" with standard Windows-style window controls (minimize, maximize, close). The window contains a registration form with the following fields and values:

Label	Value
Nome	Zuleide Maria
Cidade	Tupã
Rua	Brasil
Número	1010
Bairro	Centro

Below the form is a button labeled "Exibir Dados".

Below the button, the following text is displayed:

Olá Zuleide Maria seu endereço é Rua Brasil nº 1010
Bairro: Centro - Cidade: Tupã

At the bottom of the window, the email address "ewertonjsilva@gmail.com" is displayed.

Exercícios

21. Identifique os erros no programa abaixo.

```
implementation

{$R *.dfm}

procedure TfrmExemplo.btnNomeCompletoClick(Sender: TObject);
begin
    // Combina o nome e o sobrenome digitados nos Edits
    lblNomeCompleto := 'Nome completo: ' + edtNome.Text
                      + ' ' + edtSobrenome.Text
    { Textos inseridos via código devem ser escritos entre aspas
      simples
    end;

end.
```

Exercícios

22. Identifique o erro no programa abaixo.

```
procedure TfrmExemplo.btnNomeCompletoClick(Sender: TObject);  
begin  
    // Combina o nome e o sobrenome digitados nos Edits  
    lblNomeCompleto.Caption = 'Nome completo: ' + edtNome.Name  
                                + ' ' + edtSobrenome.Text;  
    { Textos inseridos via código devem ser escritos entre aspas  
    simples }  
end;
```

Exercícios

23. Identifique os erros no programa abaixo.

```
procedure TfrmExemplo.btnNomeCompletoClick(Sender: TObject);  
begin  
    // Combina o nome e o sobrenome digitados nos Edits  
    lblNomeCompleto.Caption := 'Nome completo:  + edtNome.Text'  
                                + ' '  edtSobrenome.Text;  
    { Textos inseridos via código devem ser escritos entre aspas  
      simples  
    end;  
  
end.
```

Bibliografia

- Ana Paula Muniz; Diogo Rocha Ferreira de Menezes. Artigo Evolução da linguagem Delphi – Web Publicação em 13/08/ 2014. Disponível em <http://pt.slideshare.net/diogorochamenezes/evolucao-da-linguagem-delphi-artigo>. Acesso em 26/04/2015
- ANSELMO, FERNANDO ANTONIO F. Desvendando o Caminho das Pedras. Web Publicação de 1995-97. Disponível em <http://www.greantoniobraga.seed.pr.gov.br/redeescola/escolas/13/870/10/arquivos/File/Adenildo/Biblia-Delphi-7-PtBr.pdf>. Acesso em 27/04/2015
- BARCELOS, ÉRICA. Linguagem de Programação Delphi Orientada a objeto – Web Publicação de Fevereiro de 2012. Disponível em <http://www.simonson.br/its/pdf/apostilas/base-tecnica/2/linguagem-de-programacao-1-capitulo-2-ano-informatica.pdf> e <http://www.simonson.br/its/pdf/apostilas/base-tecnica/2/linguagem-de-programacao-2-capitulo-2-ano-informatica.pdf>. Acesso em 12/03/2014.
- LEÃO, MARCELO. Borland Delphi 7 curso completo. Editora Axcel Books, 1ª Edição, 2003.
- PEREIRA, PAULO ROBERTO ALVES. Desenvolvendo aplicações orientadas a objetos com Borland Delphi. Web Publicação de 2002. Disponível em <http://www2.fateb.br/ftp/apostilas/Delphi/OO-Delphi.pdf>. Acesso em 19/03/2014.
- YAMAZACK, WESLEY. Recursos de Compilação no Delphi. Web Publicação. <http://www.devmedia.com.br/recursos-de-compilacao-no-delphi/16964>. Acesso em 25/01/2016